

งานปฏิบัติการกลุ่ม · กลุ่มที่ 2

Browser Navigation

Two-Stack Method vs ArrayList + Current Index

การออกแบบและวิเคราะห์อัลกอริทึมด้วย Stack โดยใช้ภาษา Java

สมาชิกกลุ่ม

นายธนโชติ กสิกิจกรกุล 68122250097

นายณัฐดนัย ลิ้มธรรมรงค์ 66122250037

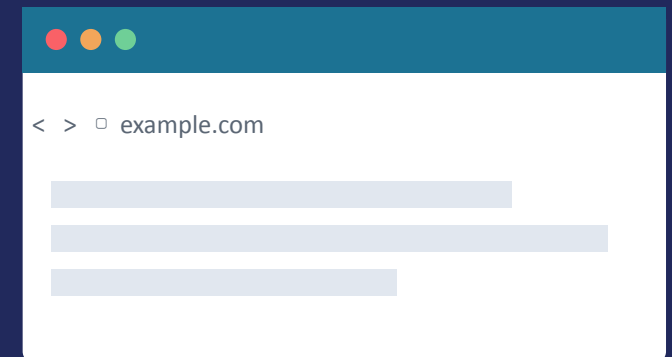
นายพงษ์นรินทร์ โคตรวงศ์ทอง 68122250015

นายภูวสิทธิ์ ปันดี 67122250042

นายมนเฑียร พุทธเสน 68122250063

นายณัฐวุฒิ ชมภูวิเศษ 67122250014

นางสาวพัชราภรณ์ ไป่สูงเนิน 67122250065



โจทย์: ระบบ Browser Navigation

สถานการณ์

- ระบบ Browser ต้องรองรับการเปิดหน้าเว็บใหม่ การย้อนกลับ (Back) และการไปข้างหน้า (Forward)
- เมื่อผู้ใช้กด Back แล้วเปิดหน้าใหม่ ประวัติของหน้า Forward ต้องถูกล้างทิ้งทันที
- ข้อมูลของแต่ละหน้า: Page ID, Page Title, URL, Visited Time

คำสั่งที่ต้องรองรับ

VISIT page

BACK

FORWARD

CURRENT

DISPLAY HISTORY

Invariant สำคัญ

เมื่อ VISIT หน้าใหม่หลังจาก Back มา ถือว่าเป็นการสร้าง “สายใหม่” (new branch) — สาย Forward เดิมจึงไม่ถูกต้องอีกต่อไป และต้องถูกล้างทิ้งทั้งหมด เหมือนพฤติกรรมจริงของ Browser ทั่วไป

VISIT A → B → C → Back → Back →
Visit D
Forward [B,C] ถูกล้างทิ้งทันทีที่ Visit D

Input, Output และ Constraints

I Input

- คำสั่ง VISIT/BACK/FORWARD/CURRENT/DISPLAY ที่ละคำสั่ง
- ข้อมูลหน้า: Page ID, Title, URL, Visited Time

O Output

- Current Page, Back History, Forward History หลังทุกคำสั่ง
- ค่า true/false ของ BACK และ FORWARD

C Constraints

- BACK ไม่ได้เมื่อไม่มีหน้าก่อนหน้า
- FORWARD ไม่ได้เมื่อไม่มีหน้าถัดไป
- VISIT ต้องล้าง Forward History ทันที
- รองรับประวัติได้ถึงระดับแสนหน้า

Two-Stack Method

แนวคิดหลัก

- backStack เก็บหน้าก่อน currentPage
- forwardStack เก็บหน้าที่ถูก Back ออกมา
- VISIT ใหม่ $\rightarrow$ push current เดิมลง backStack แล้วล้าง forwardStack ทั้งทั้งหมด
- BACK/FORWARD $\rightarrow$ สลับ push/pop ระหว่างสอง Stack

ข้อดี: BACK/FORWARD เป็น $O(1)$ เสมอ ตรงกับพฤติกรรม LIFO ตามธรรมชาติ

ข้อจำกัด: VISIT หลัง Back มาไกล ต้องล้าง forwardStack ได้ถึง $O(n)$

Pseudocode: Visit()

1. IF currentPage $\neq$ null
2. backStack.push(currentPage)
3. WHILE forwardStack not empty
4. forwardStack.pop()
5. currentPage $\leftarrow$ newPage

Time / Space Complexity

VISIT $O(1)$ amortized / $O(n)$ worst

BACK $O(1)$

FORWARD $O(1)$

Space $O(n)$

ArrayList + Current Index Method

แนวคิดหลัก

- historyList เก็บทุกหน้าของสายปัจจุบัน เรียงตามเวลา
- currentIndex ชี้ตำแหน่งหน้าปัจจุบัน
- VISIT ใหม่ → ตัด element หลัง currentIndex ทิ้ง แล้วเพิ่มหน้าใหม่ต่อท้าย
- BACK/FORWARD → ขยับ currentIndex ถอยหน้า/เดินหน้า

ข้อดี: โครงสร้างเดียว เข้าถึงหน้าใด ๆ ด้วย index ได้โดยตรง (random access)

ข้อจำกัด: การตัด list ท้าย (removeRange) มี overhead ภายในสูงกว่า Two-Stack

Pseudocode: Visit()

1. $removed \leftarrow size - (currentIndex + 1)$
2. IF $removed > 0$
3. ลบ element ท้าย list ออก
4. $historyList.add(newPage)$
5. $currentIndex \leftarrow currentIndex + 1$

Time / Space Complexity

VISIT $O(1)$ amortized / $O(n)$ worst

BACK $O(1)$

FORWARD $O(1)$

Space $O(n)$

ตัวอย่างการทำงาน (Algorithm A)

ชุดคำสั่งบังคับ: VISIT A, VISIT B, VISIT C, BACK, BACK, FORWARD, VISIT D, BACK, FORWARD

Step	คำสั่ง	Current	Back Stack	Forward Stack
1	VISIT A	A	(empty)	(empty)
2	VISIT B	B	[A]	(empty)
3	VISIT C	C	[B, A]	(empty)
4	BACK	B	[A]	[C]
5	BACK	A	(empty)	[B, C]
6	FORWARD	B	[A]	[C]
7	VISIT D	D	[B, A]	(empty) ← ล้าง Forward
8	BACK	B	[A]	[D]
9	FORWARD	D	[B, A]	(empty)

การพิสูจน์ความถูกต้อง (Algorithm A)

แนวทาง: Precondition / Postcondition ร่วมกับ Loop Invariant (ขั้นตอนล้า forwardStack)

1

Precondition

backStack/forwardStack เป็น Stack ที่ถูกต้อง (LIFO) และ currentPage เป็นหน้าปัจจุบันหรือ null

2

Invariant

ระหว่างวน pop ล้าง forwardStack element ที่เหลือยังคงลำดับเดิมครบถ้วน ไม่มีการสูญหาย

3

Postcondition

เมื่อ VISIT จบ: currentPage = newPage, forwardStack ว่าง, backStack มี current เดิมอยู่บนสุด

4

ไม่กระทบข้อมูลอื่น

push/pop กระทำที่ปลาย Stack ด้านเดียว (LIFO) จึงไม่แทรก/สลับลำดับ element อื่น

5

การยุติ

operation ทุกตัวมี push/pop จำนวนจำกัด ($\leq$ ขนาด Stack ขณะนั้น) จึงยุติเสมอ

Time & Space Complexity ของทุกฟังก์ชัน

Function	Best	Average	Worst	Aux. Space
VISIT	$O(1)$	$O(1)$ amortized	$O(n)$	$O(1)$ /ครั้ง
BACK	$O(1)$	$O(1)$	$O(1)$	$O(1)$
FORWARD	$O(1)$	$O(1)$	$O(1)$	$O(1)$
CURRENT	$O(1)$	$O(1)$	$O(1)$	$O(1)$
DISPLAY HISTORY	$O(1)$	$O(n)$	$O(n)$	$O(n)$

ทำไม VISIT เป็น $O(1)$ amortized?

แต่ละหน้าใน forwardStack ถูก pop ทิ้งได้อย่างมาก
 หนึ่งครั้งก่อนหายไปตลอดกาล งานรวมตลอดลำดับ
 operation จึงแปรผันเชิงเส้นกับจำนวน operation
 ทั้งหมด ทำให้ต้นทุนเฉลี่ยต่อครั้งคือ $O(1)$

ห้ามสรุปแค่ $push()/pop() = O(1)$ ต้องวิเคราะห์ทุกฟังก์ชัน
 ของระบบ

โครงสร้างคลาสของโปรแกรม

Page

Data Model: pageId, title, url, visitedTime

BrowserHistory

«interface»

visit(), back(), forward(), current(), displayHistory()

TwoStackHistory

Algorithm A — ArrayDeque×2 (backStack, forwardStack)

ArrayListHistory

Algorithm B — ArrayList + currentIndex

Main

เมนูรับคำสั่ง, Input Validation, SWITCH A|B

TestCases / ExperimentRunner

รัน 8 Test Cases และการทดลองประสิทธิภาพ

BrowserHistory (interface) → นำไปใช้โดย TwoStackHistory และ ArrayListHistory (polymorphism) เพื่อให้ Main เรียกใช้งานแบบเดียวกัน และเปรียบเทียบผลลัพธ์ได้อย่างยุติธรรม

ผลการทดสอบ 8 กรณีบังคับ

#	Test Case	A: Two-Stack	B: ArrayList
1	Back เมื่อไม่มีหน้าก่อนหน้า	PASS	PASS
2	Forward เมื่อไม่มีหน้าถัดไป	PASS	PASS
3	เปิดหน้าแรก	PASS	PASS
4	เปิดหน้าเดิมซ้ำ	PASS	PASS
5	Back หลายครั้ง	PASS	PASS
6	Forward หลายครั้ง	PASS	PASS
7	Back แล้วเปิดหน้าใหม่ (ล้าง Forward)	PASS	PASS
8	ประวัติจำนวนมาก (n=20,000)	PASS	PASS

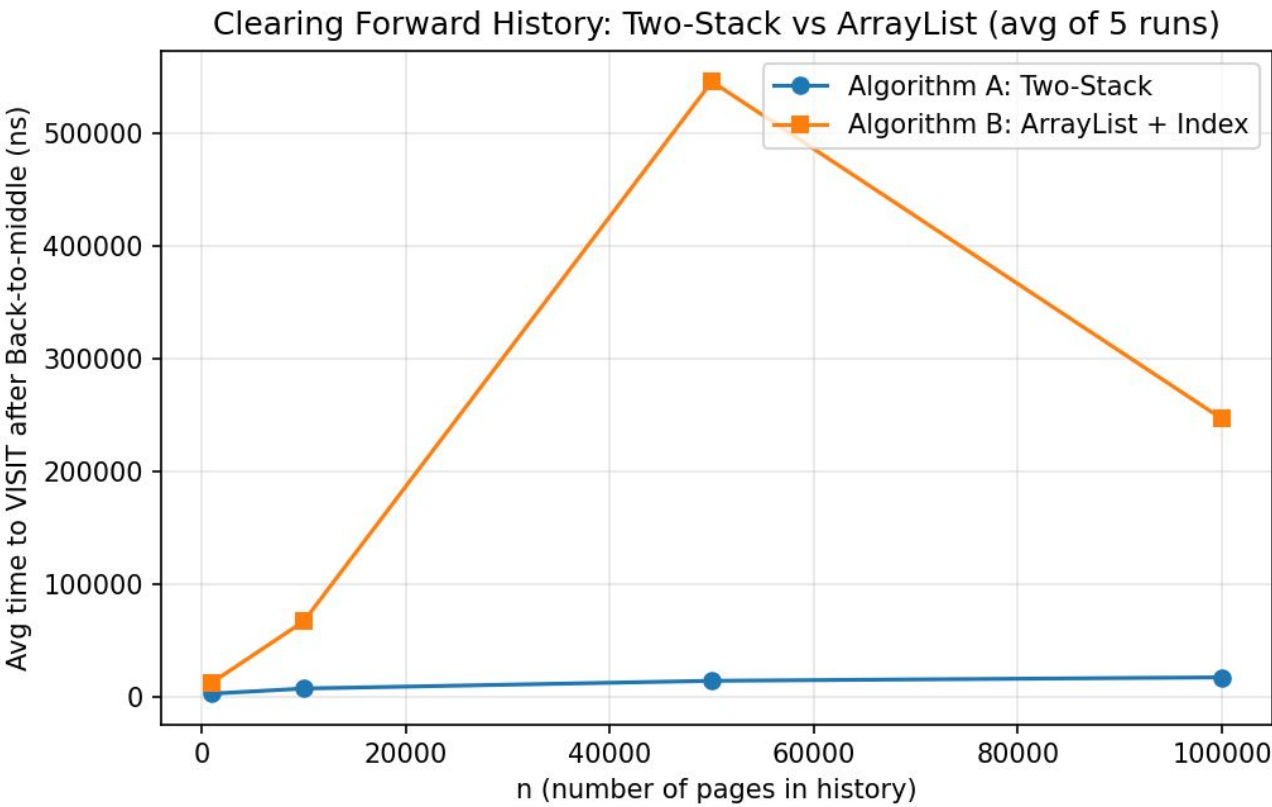
18 / 18

PASS ทั้งหมด

(8 กรณี × 2 อัลกอริทึม + การตรวจสอบสถานะเริ่มต้น)

ผลการทดลองประสิทธิภาพ

สร้างประวัติ n หน้า $\rightarrow$ Back ไปกึ่งกลาง $\rightarrow$ VISIT หน้าใหม่ $\rightarrow$ วัดเวลาล้าง Forward History (เฉลี่ย 5 รอบ)



n	A (ns)	B (ns)
1,000	3,028	12,934
10,000	7,594	67,297
50,000	14,434	546,381
100,000	17,505	247,073

Two-Stack เร็วกว่า ArrayList 4-35 เท่า แม้ทั้งคู่จะเป็น $O(n)$ เท่ากัน — Big-O บอกอัตราการเติบโต แต่ไม่บอกค่าคงที่ (constant factor) ของการทำงานจริง

สรุปและวิธีที่กลุ่มเลือก

- ทั้งสองอัลกอริทึมแก้ปัญหเดียวกันและให้ผลลัพธ์ถูกต้องตรงกันทุก Test Case (18/18 PASS)
- Time Complexity ระดับเดียวกัน: $O(1)$ amortized สำหรับ VISIT, $O(1)$ worst-case สำหรับ BACK/FORWARD
- ผลทดลองจริงแสดงว่า Two-Stack มีค่าคงที่ (constant factor) ต่ำกว่าอย่างชัดเจนในทุกขนาดข้อมูล

วิธีที่กลุ่มเลือก: Two-Stack Method

เหมาะสมกว่าสำหรับ Browser History เพราะโครงสร้างสอดคล้องกับพฤติกรรม Back/Forward แบบ LIFO โดยตรง และให้เวลาทำงานจริงเร็วอย่างสม่ำเสมอ

ไฟล์ที่ต้องส่ง

- Source Code Java + GitHub Repository
- รายงาน PDF และสไลด์นำเสนอ
- Pseudocode, Flowchart, Class Diagram
- Test Case Table + ไฟล์ผลการทดลอง/กราฟ
- AI Usage Log
- รายงานการแบ่งงานของสมาชิก